

0.33. Écrire soi-même sa fonction

`mon_deplace_x`, écrite à la main : le 0.17 rangé dans une fonction. C'est ce qu'il y a dans `deplace_x`.

1. Ce qu'il faut comprendre

Toutes ces fonctions sont écrites dans le même langage que tes programmes. Ici, on écrit la nôtre, `mon_deplace_x` : c'est, ligne pour ligne, ce que la console a dans `deplace_x`.

Ce qui est nouveau ici : écrire une fonction. `uint8_t mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas) { ... }`. Le premier `uint8_t` dit ce que la fonction **rend** (un octet, la colonne) ; entre les parenthèses, ses **paramètres**, ce qu'elle reçoit ; entre les accolades, ce qu'elle fait.

Les paramètres sont des copies. Quand on appelle `mon_deplace_x(x, 0, ALPHABET[0], 5)`, `colonne` reçoit une copie de `x` (0), `pas` reçoit 5. La fonction change `colonne` et `pas` tant qu'elle veut : `x`, lui, ne bouge pas.

`return colonne;` rend la colonne d'arrivée, et termine la fonction. C'est cette valeur que `x = ...` range dans `x`.

Ici, `x =` est obligatoire. Le rangement automatique du 0.23 ne vaut que pour les fonctions de la console. `mon_deplace_x(x, ...);` seule ferait bouger la lettre, mais `x` resterait à 0 : c'est la règle normale du C.

`break` sort de la boucle `while` tout de suite : au bord de l'écran, il n'y a plus de pas possible.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1077 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          52 o ← la console, à l'allumage — la mise en route de la console
mon_deplace_x()    156 o ← main() — ta fonction
main()             47 o ← démarrage — ta fonction
VBlank              8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage       28 o ← image() — attend l'image suivante
AttendreVBlank      16 o ← démarrage — attend que l'écran soit libre
AttendreAcces       12 o ← EffacerCases — attend de pouvoir écrire une case
AdresseCase         10 o ← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases        12 o ← effacer() — efface des cases
CopierTuiles        18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte        14 o ← démarrage — vide les deux cartes de l'écran
Tuiles              704 o ← CopierTuiles — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
```

DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser()      × 2    26 o sur place, dans mon_deplace_x() + AdresseCase (10 o)
effacer()    × 1    15 o sur place, dans mon_deplace_x() + AdresseCase (10 o), EffacerCases (12 o)
image()      × 2     6 o sur place, dans mon_deplace_x(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
mon_deplace_x() × 2    36 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <poser>      // pose une tuile sur une case du fond
#include <effacer>     // efface des cases, ou tout le fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — X

```
uint8_t x = 0;        // la colonne de la lettre

// NOTRE fonction : c'est le 0.17, rangé sous un nom.
//
//  uint8_t      mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas)
//  |            |            |
//  |            |            +-- les PARAMÈTRES : ce qu'elle reçoit. Des COPIES :
//  |            |            changer colonne ne change pas x.
//  |            +----- son nom
//  +----- ce qu'elle REND : un octet (la colonne d'arrivée)
```

3. UNE FONCTION — MON_DEPLACE_X()

```
uint8_t mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas) {
    uint8_t recule = pas > 127;        // -5 est rangé 251 : au-delà de 127, on recule
    if (recule) pas = 0 - pas;         // 251 redevient 5 : le nombre de cases
    poser(colonne, ligne, tuile);     // la lettre à sa place de départ
    while (pas > 0) {                  // tant qu'il reste des pas
        for (uint8_t i = 0; i < 15; i++) image(); // un quart de seconde
        if (recule && colonne == 0) break;        // bord gauche : on sort de la boucle
        if (!recule && colonne == 19) break;      // bord droit : on sort de la boucle
        effacer(colonne, ligne, 1);             // 1. efface l'ancienne place
        if (recule) colonne--;                   // 2. une colonne plus à gauche...
        else colonne++;                          // ...ou plus à droite
        poser(colonne, ligne, tuile);           // 3. la lettre à sa nouvelle place
        pas--;                                  // un pas de moins à faire
    }
    return colonne;                        // REND la colonne d'arrivée, et termine
}
```

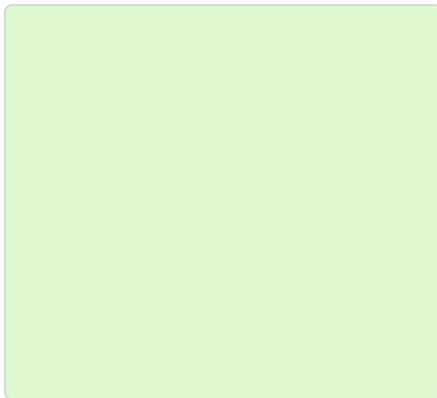
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // « x = » est OBLIGATOIRE ici : mon_deplace_x est à nous, pas à la console.
    // Elle reçoit une copie de x, et REND la colonne d'arrivée ; on la range.
    x = mon_deplace_x(x, 0, ALPHABET[0], 5);    // x vaut 5
    x = mon_deplace_x(x, 0, ALPHABET[0], -5);   // x vaut 0

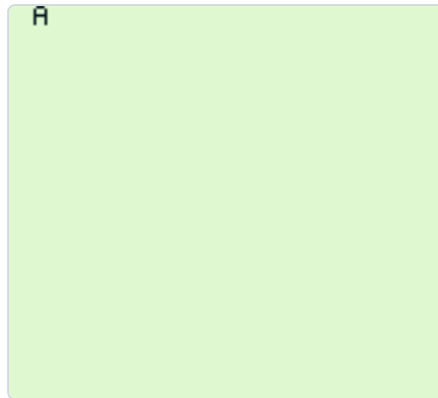
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

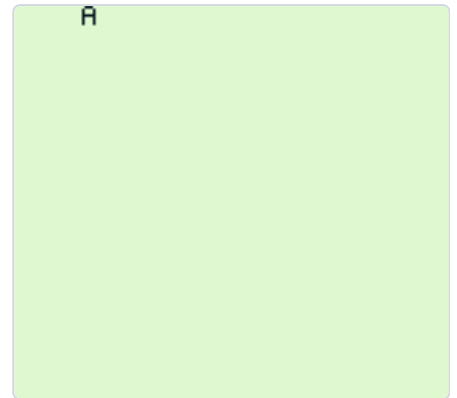
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



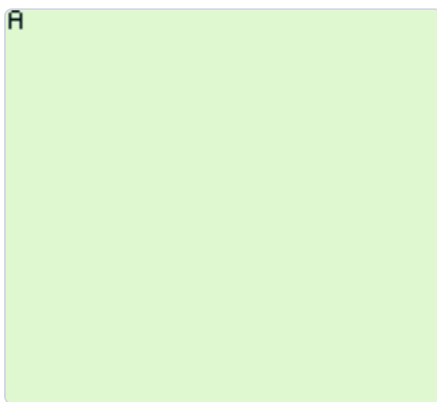
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Exactement le 0.19 : un A qui avance de 5 cases, revient, et s'arrête. Mais cette fois, la fonction est écrite dans le programme.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ à l'aller, le A arrive en colonne 5 (« A »)

✓ au retour, le A revient en colonne 0 (« A »)

✓ x vaut la colonne rendue ($x = 0$)

Ensuite : **0.33.1. Écrire soi-même sa fonction — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++